CHAPTER -4

FUNCTIONAL DEPENDENCIES AND ANALYZE REDUNDANCY AND DATA ANOMALIES

INTRODUCTION

Functional Dependencies

A Functional Dependency (FD) describes the relationship between attributes in a database. It states that the value of one attribute (or a set of attributes) uniquely determines the value of another attribute. Functional dependencies help identify the correct primary keys and are essential for normalizing a database to reduce redundancy and maintain data consistency.

Why Functional Dependencies are Important

- Identify Primary Keys
- Reduce Data Redundancy
- Avoid Data Anomalies
- Improve Data Integrity
- Support Database Normalization
- Ensure Efficient Database Design

1. CUSTOMER TABLE

Functional Dependency

Customer_ID → Customer_Name, Email, Phone_Number, Address

Redundancy Analysis

Customer information is stored only once in the Customer table. Orders store only the Customer_ID, preventing duplicate customer records.

Data Anomalies

- Insert Anomaly: No Anomaly. A new customer can be added without creating an order.
- Update Anomaly: No Anomaly. Customer details are stored only in the Customer table, so updates are made in one place.
- Delete Anomaly: No Anomaly. Deleting an order will not remove customer information.

Conclusion

The Customer table is free from insert, update, and delete anomalies because customer information is stored independently and referenced through the Customer_ID.

2. SELLER TABLE

Functional Dependency

Seller_ID → Seller_Name, Store_Name, Email, Phone_Number, Address

Redundancy Analysis

Seller information is stored only once in the Seller table. Products store only the Seller_ID, preventing duplicate seller records.

Data Anomalies

- Insert Anomaly: No Anomaly. A new seller can be registered without adding a product.
- Update Anomaly: No Anomaly. Updating seller information requires changes only in the Seller table.
- Delete Anomaly: No Anomaly. Deleting a product will not remove seller information.

Conclusion

The Seller table avoids data redundancy and maintains seller information independently.

3. CATEGORY TABLE

Functional Dependency

Category_ID → Category_Name, Category_Description

Redundancy Analysis

Category details are stored only once in the Category table. Products reference Category_ID instead of repeating category information.

Data Anomalies

- Insert Anomaly: No Anomaly. A new category can be added without creating products.
- Update Anomaly: No Anomaly. Updating a category name requires changes only in the Category table.
- Delete Anomaly: No Anomaly. Deleting a product does not remove category information.

Conclusion

The Category table maintains unique category information and eliminates duplicate records.

4. PRODUCT TABLE

Functional Dependency

Product_ID → Product_Name, Description, Price, Stock, Image, Seller_ID, Category_ID

Redundancy Analysis

Product details are stored only once. Seller and category details are maintained using foreign keys.

Data Anomalies

- Insert Anomaly: Exists. A product cannot be inserted unless a valid Seller_ID and Category_ID already exist.
- Update Anomaly: No Anomaly. Product information is stored only in the Product table.

- Delete Anomaly: No Anomaly. Deleting a product will not remove seller or category information.

Conclusion

The Product table maintains data consistency by using foreign keys and avoids redundant product information.

5. CART TABLE

Functional Dependency

Cart_ID → Customer_ID, Product_ID, Quantity

Redundancy Analysis

The Cart table stores only product references and quantity, reducing duplicate customer and product information.

Data Anomalies

- Insert Anomaly: Exists. A cart item can be added only if the customer and product already exist.
- Update Anomaly: No Anomaly. Updating quantity requires changes only in the Cart table.
- Delete Anomaly: No Anomaly. Removing a cart item does not affect customer or product information.

Conclusion

The Cart table efficiently manages shopping cart data while maintaining referential integrity.

6. ORDER TABLE

Functional Dependency

Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status

Redundancy Analysis

The Order table stores only order details. Customer information is referenced using Customer_ID.

Data Anomalies

- Insert Anomaly: Exists. An order can be created only for an existing customer.
- Update Anomaly: No Anomaly. Order information is stored only in the Order table.
- Delete Anomaly: No Anomaly. Deleting an order does not remove customer information.

Conclusion

The Order table maintains accurate transaction records while preserving customer data.

7. ORDER_ITEM TABLE

Functional Dependency

Order_Item_ID → Order_ID, Product_ID, Quantity, Price

Redundancy Analysis

Order items store only references to orders and products, avoiding duplicate information.

Data Anomalies

- Insert Anomaly: Exists. An order item requires a valid Order_ID and Product_ID.
- Update Anomaly: No Anomaly. Quantity and price are updated only in the Order_Item table.
- Delete Anomaly: No Anomaly. Deleting an order item does not affect the order or product.

Conclusion

The Order_Item table maintains accurate product details for each order while preventing data duplication.

8. PAYMENT TABLE

Functional Dependency

Payment_ID → Order_ID, Payment_Method, Payment_Status, Payment_Date

Redundancy Analysis

Payment information is stored separately and linked using Order_ID.

Data Anomalies

- Insert Anomaly: Exists. Payment information can be recorded only after an order is created.
- Update Anomaly: No Anomaly. Payment details are updated only in the Payment table.
- Delete Anomaly: No Anomaly. Deleting a payment record does not remove order information.

Conclusion

The Payment table ensures secure transaction management while maintaining data consistency.

9. SHIPMENT TABLE

Functional Dependency

Shipment_ID → Order_ID, Delivery_Address, Shipment_Status, Delivery_Date

Redundancy Analysis

Shipment information is maintained separately and linked through Order_ID.

Data Anomalies

- Insert Anomaly: Exists. Shipment details can be added only for an existing order.
- Update Anomaly: No Anomaly. Shipment status is updated only in the Shipment table.

- Delete Anomaly: No Anomaly. Deleting shipment information does not remove order records.

Conclusion

The Shipment table supports efficient delivery tracking while maintaining data integrity.

10. REVIEW TABLE

Functional Dependency

Review_ID → Customer_ID, Product_ID, Rating, Review_Text, Review_Date

Redundancy Analysis

Reviews are stored separately and linked using Customer_ID and Product_ID.

Data Anomalies

- Insert Anomaly: Exists. A review can be added only for an existing customer and product.
- Update Anomaly: No Anomaly. Review details are stored only in the Review table.
- Delete Anomaly: No Anomaly. Deleting a review does not remove customer or product information.

Conclusion

The Review table stores customer feedback independently while preserving related customer and product records.

11. INVENTORY TABLE

Functional Dependency

Inventory_ID → Seller_ID, Product_ID, Stock_Quantity, Last_Updated

Redundancy Analysis

Inventory information is stored separately and linked to sellers and products.

Data Anomalies

- Insert Anomaly: Exists. Inventory records require valid Seller_ID and Product_ID.
- Update Anomaly: No Anomaly. Stock quantity is updated only in the Inventory table.
- Delete Anomaly: No Anomaly. Deleting inventory records does not remove seller or product information.

Conclusion

The Inventory table provides accurate stock management while maintaining relationships between sellers and products.

12. ADMINISTRATOR TABLE

Functional Dependency

Admin_ID → Admin_Name, Email, Phone_Number

Redundancy Analysis

Administrator information is stored only once in the Administrator table.

Data Anomalies

- Insert Anomaly: No Anomaly. A new administrator can be added independently.
- Update Anomaly: No Anomaly. Administrator details are stored only in the Administrator table.
- Delete Anomaly: No Anomaly. Deleting an administrator does not affect customer, seller, product, or order data.

Conclusion

The Administrator table maintains administrative information independently and is free from insert, update, and delete anomalies.